Zero-Shot Text Classification with Locally-Hosted Large Language Models: The `ollama-classifier` Library

Luigi Palumbo* Mengting Yu Carolina Camassa

June 16, 2026

Abstract

Large Language Models (LLMs) have emerged as powerful zero-shot classifiers, yet their adoption in domains that demand data privacy, cost control, and offline operation remains limited by reliance on proprietary cloud APIs. We present `ollama-classifier`, an open-source Python library that enables zero-shot and few-shot text classification using locally-hosted LLMs through the Ollama runtime. The library provides constrained output generation, confidence scoring, batch processing, and integration with multiple inference backends (Ollama, vLLM, SGLang, llama.cpp), making LLM-based classification accessible without fine-tuning, API keys, or internet connectivity. A companion desktop GUI built with Flet allows non-technical users to leverage retrieval-augmented classification through an intuitive workflow; the application is publicly released as `ollama-classifier-gui` on GitHub and the Python Package Index (PyPI). We benchmark the library against BART-large-MNLI and scikit-llm on a COICOP product classification task, demonstrating that a compact 3B-parameter model served locally substantially outperforms the NLI-based baseline and matches scikit-llm in accuracy while additionally providing calibrated confidence scores that reliably discriminate correct from incorrect predictions.

Keywords: zero-shot classification, large language models, local inference, retrieval-augmented classification, Ollama, open-source software

1 Introduction

The advent of Large Language Models (LLMs) has fundamentally transformed natural language processing, enabling remarkable zero-shot and few-shot performance across a wide range of tasks (Brown et al., 2020; Radford et al., 2019; Zhao et al., 2023). Among these tasks, text classification—the assignment of predefined categorical labels to free-form text—stands out as a particularly impactful application, with use cases spanning sentiment analysis (Z. Wang et al., 2024), economic taxonomy coding, and product categorization (Nunes et al., 2025).

Traditionally, text classification has relied on supervised machine learning pipelines that require substantial labeled training data, feature engineering, and model fine-tuning. While transformer-based architectures such as BERT (Berki et al., 2025) have improved classification accuracy, they remain data-intensive and offer limited adaptability to new or evolving taxonomies without costly retraining. The emergence of LLMs has challenged this paradigm: models such as GPT-3 (Brown et al., 2020) and GPT-4 (OpenAI, 2023) have demonstrated that generative models can perform classification through in-context learning, eliminating the need for task-specific training data.

However, leveraging these capabilities in production environments—particularly in sensitive domains such as official statistics, healthcare, and finance—introduces significant challenges.

*Corresponding author.

Proprietary cloud-based APIs incur recurring costs, require internet connectivity, and raise data privacy concerns when processing confidential or regulated information. These constraints limit the applicability of LLM-based classification in settings where data sovereignty and operational independence are paramount.

The recent proliferation of open-source LLMs—including LLaMA (Meta AI, 2024; Touvron et al., 2023), Mistral (Jiang et al., 2023), Qwen (Yang et al., 2024), and DeepSeek-R1 (DeepSeek-AI, 2025)—combined with lightweight serving frameworks such as Ollama (Ollama Contributors, 2023), has made it feasible to run capable language models on consumer hardware. This shift creates an opportunity to build classification tools that combine the zero-shot reasoning power of LLMs with the privacy, cost-effectiveness, and offline capability of local inference.

We present `ollama-classifier`, an open-source Python library that realizes this opportunity by providing a unified interface for zero-shot and few-shot text classification using locally-hosted LLMs. The library is designed around three core principles:

1. **No fine-tuning required.** Classification is performed through prompting and retrieval augmentation, leveraging the model's pre-trained knowledge without any task-specific training.

2. **Local and private.** All inference runs on the user's hardware through the Ollama runtime or compatible backends, ensuring that no data leaves the local machine.

3. **Accessible to non-technical users.** A companion GUI application provides a point-and-click interface for the full classification workflow, abstracting away the complexity of prompt engineering, backend configuration, and batch processing.

The remainder of this paper is organized as follows. Section 2 reviews related work on LLM-based classification and retrieval augmentation. Section 3 provides background on the enabling technologies. Section 4 describes the `ollama-classifier` library's architecture and API. Section 5 introduces the companion GUI application. Section 6 illustrates a real-world application to product classification and presents a benchmark evaluation comparing `ollama-classifier` against a BART-large-MNLI zero-shot baseline and the scikit-llm library. Section 7 discusses design decisions and limitations, and Section 8 concludes with directions for future work.

2 Related Work

2.1 LLMs as Classifiers

The use of generative language models as classifiers has its roots in the observation that autoregressive LLMs can perform discriminative tasks through careful prompt design. Radford et al. (2019) first demonstrated that GPT-2 could perform zero-shot task transfer without explicit supervision, establishing the paradigm of leveraging generative pre-training for discriminative downstream tasks. This capability was dramatically scaled by GPT-3 (Brown et al., 2020), which showed that few-shot in-context learning could achieve competitive performance on classification benchmarks without any gradient updates.

Subsequent work has explored the boundaries of this approach. Liu et al. (2023) provide a systematic survey of prompting methods, cataloguing strategies for instructing LLMs to perform classification through template-based prompts. Mu and Ziebart (2024) investigate how prompt complexity affects zero-shot classification performance, finding that well-structured prompts with explicit instructions and output constraints significantly improve accuracy. Gilardi et al. (2023) demonstrate that ChatGPT can match or exceed crowd-worker quality on text annotation tasks, providing empirical support for using LLMs as practical classification tools.

2.2 Retrieval-Augmented Classification

Retrieval-Augmented Generation (RAG), introduced by P. Lewis et al. (2020), grounds LLM outputs in externally retrieved evidence, improving both factuality and stability. This paradigm has been adapted to classification tasks through Retrieval-Augmented Classification (RAC). Yu et al. (2023) demonstrate that combining nearest-neighbor retrieval with few-shot prompting improves text classification without fine-tuning. Abdullahi et al. (2024) extend this to the zero-shot setting, showing that retrieved exemplars can enable classification into arbitrary taxonomies without training data.

Most recently, Nunes et al. (2025) apply RAC to COICOP 2018 product classification, constructing a synthetic knowledge base of 34,451 exemplars and achieving up to 0.80 accuracy at the finest taxonomy level. The `ollama-classifier` library builds directly on this line of research, operationalizing the RAC framework as a general-purpose, reusable Python tool.

2.3 Existing Classification Libraries

Several open-source projects address LLM-based text classification. `llmclassifier` (CVxTz, 2023) provides a Python framework for building classification pipelines with LLMs, but targets API-based models and does not support local inference. `llmClassificR` (Golino, 2023) offers similar functionality in R, again relying on cloud-hosted models. `scikit-llm` (Kondrashchenko & Kostromin, 2023) provides a scikit-learn-compatible interface for zero-shot and few-shot classification with LLMs and can be directed at a local model server through an OpenAI-compatible endpoint. Unlike `ollama-classifier`, scikit-llm is a label-only classifier: it returns the predicted class but does not expose calibrated confidence scores or a full probability distribution over the candidate labels, limiting its use for uncertainty-aware triage workflows.

`ollama-classifier` differentiates itself by (i) focusing exclusively on local inference through Ollama and compatible backends, (ii) providing constrained output generation that forces the model to return only valid class labels, (iii) implementing confidence scoring through multi-call softmax evaluation, and (iv) offering a dedicated GUI application for non-technical users—features not found together in any existing library.

To extend the `ollama-classifier` ecosystem beyond Python, we developed `rollama-classifier` (Palumbo 2026b), a native R package that ports the library's constrained-output classification and multi-backend support to the R environment. This enables R users—particularly in statistical offices, government agencies, and academic research—to leverage the same LLM-based classification capabilities without leaving their preferred analysis environment.

3 Background

3.1 The Ollama Runtime

Ollama (Ollama Contributors, 2023) is an open-source framework for running LLMs locally on consumer hardware. It provides a lightweight model server with a REST API, model management (pull, list, delete), and support for quantized inference that enables running models ranging from 1B to 70B+ parameters on standard workstations.

The Ollama Python library (Ollama Team, 2023) provides a convenient client interface, handling API communication and structured outputs. The `ollama-classifier` library wraps this client to offer a higher-level classification API, abstracting away prompt construction, output parsing, and confidence computation.

3.2 Inference Backends

While Ollama serves as the default backend, `ollama-classifier` supports multiple inference engines through a unified interface: Ollama for consumer hardware and single-user workflows;

vLLM for high-throughput batch inference; SGLang for low-latency serving; and llama.cpp for resource-constrained CPU and mixed CPU/GPU environments. All backends share a common configuration interface, allowing users to switch between them without modifying application code.

3.3 Embedding Models for Retrieval

The retrieval component of RAC relies on embedding models that convert text into dense vector representations. SentenceTransformers (Reimers & Gurevych, 2019) provides the foundational Siamese BERT architecture for producing semantically meaningful sentence embeddings. The E5 family (L. Wang et al., 2022) offers multilingual semantic similarity across 100 languages, while the GTE family (Chen et al., 2024) provides instruction-tuned embeddings with strong multilingual performance. In the RAC evaluation by Nunes et al. (2025), the GTE models achieved the highest retrieval presence (90.7%), confirming their effectiveness for classification-oriented retrieval tasks.

4 The `ollama-classifier` Library

4.1 Design Philosophy

`ollama-classifier` is designed as a lightweight, modular Python library that enables users to perform text classification with LLMs through a minimal API surface. The design follows these principles:

1. **Simplicity**: A classification call requires only a text input and a list of candidate labels. All complexity—prompt construction, output parsing, backend communication—is handled internally.

2. **Flexibility**: Users can customize the system prompt, provide label descriptions, choose between multiple classification strategies, and switch inference backends transparently.

3. **Reliability**: Constrained output generation ensures that the model returns only valid class labels, preventing hallucinated or malformed predictions. Confidence scoring enables triage between high-confidence auto-accepted labels and low-confidence cases requiring human review.

4. **Scalability**: Batch and async APIs enable processing large datasets efficiently, with concurrent requests supported across all backends.

4.2 Core API

The library exposes two primary classifier classes: `OllamaClassifier` (Ollama-specific) and `LLMClassifier` (backend-agnostic). Both share an identical API surface, shown in Table 1.

4.3 Classification Strategies

`ollama-classifier` supports two classification strategies, each offering a different trade-off between speed and confidence calibration:

1. **Generate only** (`generate`): The model receives a single prompt containing the text and candidate labels, and returns one predicted label. This is the fastest strategy, suitable when speed is critical and confidence estimation is not required.

Table 1: Core methods of the `ollama-classifier` API. Each method has an asynchronous counterpart prefixed with `a` (e.g., `agenerate`, `aclassify`).

Method	Description
`generate()`	Constrained output only; returns the predicted label without confidence scores (fastest method).
`classify()`	Full classification with confidence scores; makes N API calls for N choices to compute calibrated probabilities via softmax.
`batch_generate()`	Batch constrained output for multiple texts.
`batch_classify()`	Batch classification with confidence scores.

2. **Full classification** (`classify`): Makes N separate API calls for N candidate labels, computing raw log-probability scores that are normalized using a softmax function to produce a calibrated probability distribution over all classes:

$$\mathrm{P}(y_i \mid x) = \frac{\exp(s_i)}{\sum_{j=1}^{N} \exp(s_j)},$$

where s_i is the log-probability score for class y_i and N is the number of candidate classes. Returns both the predicted label and the full probability distribution, recommended for applications requiring both a prediction and a measure of certainty for triage purposes.

4.4 Constrained Output Generation

A key design feature of `ollama-classifier` is the enforcement of constrained output. The system prompt instructs the model to return only the exact text of one of the provided candidate labels, preventing hallucinated categories, partial responses, or verbose explanations. When supported by the backend, JSON schema constraints are applied to structurally guarantee valid output. For backends that do not support schema constraints (e.g., standard Ollama), the prompt-based approach ensures reliable output formatting through careful instruction design informed by the prompting literature (Liu et al., 2023; Mu & Ziebart, 2024).

4.5 Confidence Scoring and Human-in-the-Loop

The confidence scoring mechanism enables a human-in-the-loop workflow (Hakimi, Bowen, et al., 2026; Schroeder et al., 2025). High-confidence predictions can be automatically accepted, while low-confidence predictions are flagged for human review. Vasquez Ferrer, Turgut, et al. (2026) emphasize the importance of calibrating LLM confidence scores for automated assessment, and the multi-call softmax approach implemented in `ollama-classifier` provides a principled foundation for such calibration.

The library also supports an "opt-out" mechanism, where an additional "none of the above" option is included among the candidate labels. This allows the model to abstain from classification when none of the provided options are appropriate, reducing the risk of forcing incorrect labels (Nunes et al., 2025). In the evaluation by Nunes et al. (2025), the DeepSeek-R1 models demonstrated particularly well-calibrated opt-out behavior, with correct abstention rates exceeding 85%.

4.6 Usage Example

Listing 1 demonstrates a basic classification workflow using `ollama-classifier`. The example shows how to initialize the classifier, define candidate labels with descriptions, and classify a

product description.

Listing 1: Basic usage of `ollama-classifier` for zero-shot product classification.

```python
from ollama_classifier import OllamaClassifier

classifier = OllamaClassifier(model="llama3.2")

choices = {
    "Food and non-alcoholic beverages":
        "Edible items, fresh or packaged food, drinks",
    "Clothing and footwear":
        "Garments, shoes, accessories for personal use",
    "Electronics":
        "Consumer electronic devices and accessories",
}

result = classifier.classify(
    text="Organic whole wheat pasta 500g",
    choices=choices,
)
print(result)  # ClassificationResult(label=..., scores={...})
```

5 Graphical User Interface

While the Python library provides a powerful programmatic interface, many potential users of LLM-based classification—domain experts, statisticians, data officers, and analysts—may lack programming expertise. Amershi et al. (2019) identify the gap between ML models and end-users as a critical barrier to adoption, He et al. (2021) demonstrate that automating ML workflows significantly lowers this barrier, and Lazaridou et al. (2023) argue for democratizing access to AI tools for domain specialists.

To address this challenge, we developed `ollama-classifier-gui`, a companion desktop application that provides a point-and-click interface for the full classification workflow. The application is built with Flet, a Python UI framework that enables cross-platform desktop applications from a single codebase. The application is publicly released as open-source software: the source code is hosted on GitHub,[1] and the package is distributed through the Python Package Index (PyPI),[2] where it can be installed directly with `pip install ollama-classifier-gui` (Palumbo, 2026a).

5.1 Architecture

The GUI application wraps the `ollama-classifier` library as a backend engine, exposing its full functionality through an intuitive visual interface. The application supports the same multiple inference backends as the library (Ollama, vLLM, SGLang, llama.cpp), with all backend configuration managed through a dedicated settings panel. API keys, where required, are stored using the operating system's native secure storage mechanism (Keychain on macOS, Credential Manager on Windows, libsecret on Linux), ensuring that sensitive credentials are never stored in plain text.

5.2 Four-Tab Workflow

The application is structured around a four-tab workflow that mirrors the classification pipeline: a **Settings** tab for configuring the inference backend, model, and connection parameters; a **Data**

[1] https://github.com/paluugi/ollama-classifier-gui
[2] https://pypi.org/project/ollama-classifier-gui/

Input tab for loading text data from CSV or JSON files with column mapping and preview; a **Schema** tab for defining candidate labels and descriptions, selecting the classification strategy, and configuring custom system prompts; and a **Results** tab for viewing predicted labels and confidence scores in a structured table, with export to CSV or JSON. This workflow enables non-technical users to perform classification without encountering code or command-line interfaces.

5.3 Significance

The GUI application serves a critical role in the `ollama-classifier` ecosystem by bridging the gap between the technical capabilities of modern LLMs and the operational needs of domain practitioners. National Statistical Institutes, research teams, and government agencies that need to classify large volumes of text data—such as web-scraped product descriptions for Consumer Price Index compilation (Cavallo, 2013, 2017; Nunes et al., 2025)—can now leverage state-of-the-art LLM classification without requiring software engineering expertise.

6 Application to Product Classification

To illustrate the practical utility of `ollama-classifier`, we describe its application to the classification of consumer products into the COICOP 2018 taxonomy (United Nations Statistics Division, 2018), a task of direct relevance to the compilation of Consumer Price Indices by National Statistical Institutes.

6.1 The Classification Challenge

COICOP 2018 is an international standard developed by the United Nations to categorize household expenditures by function. The taxonomy comprises a four-level hierarchical structure, from broad divisions (e.g., "Food and non-alcoholic beverages") down to fine-grained subclasses (e.g., "Other edible products"), encompassing 294 subclasses at the most granular level. Classifying web-scraped product descriptions into this taxonomy presents several challenges: high linguistic variability across retailers and countries, strong class imbalance, textual overlap between semantically similar subclasses, and the need for multilingual coverage.

6.2 Retrieval-Augmented Classification Pipeline

`ollama-classifier` supports a full retrieval-augmented classification pipeline. The workflow proceeds in three stages:

1. **Knowledge base construction**: Synthetic exemplars are generated for each COICOP subclass using LLM prompts informed by official definitions, then embedded using models such as GTE (Chen et al., 2024) or E5 (L. Wang et al., 2022).

2. **Retrieval**: Given a product description, the system retrieves the top-k most semantically similar exemplars and their associated COICOP labels.

3. **Generative classification**: The retrieved context is incorporated into the classification prompt, and the LLM performs the final classification by reasoning over both the input text and the retrieved evidence.

This pipeline, evaluated by Nunes et al. (2025), achieves up to 0.80 accuracy at the finest COICOP level using the classification-by-definition approach with Qwen2.5-32B (Yang et al., 2024). Notably, even compact models such as Mistral 7B (Jiang et al., 2023) achieve strong overall performance (63.8% accuracy at Level 4), demonstrating that effective classification does not require the largest available models.

6.3 Deployment Scenarios

`ollama-classifier` supports two primary deployment scenarios for this application:

- **Programmatic pipeline**: Data scientists and engineers can integrate `ollama-classifier` into automated data processing pipelines, using the batch and async APIs to classify large volumes of products efficiently.
- **GUI-assisted workflow**: Non-technical users can load product datasets, define or import COICOP subclass labels, configure the classification parameters, and review results through the GUI application, with the option to export classified data for downstream analysis.

In both scenarios, the local inference model ensures that product data—which may include commercially sensitive pricing information—never leaves the user's machine.

6.4 Experimental Evaluation

To assess the practical performance of `ollama-classifier` on a fine-grained product classification task, we conducted a benchmark comparison against BART-large-MNLI (M. Lewis et al., 2019; Williams et al., 2018), a widely-used zero-shot classifier that frames text classification as natural language inference (NLI).

6.4.1 Setup

We evaluated both classifiers on a manually-labeled dataset of 637 consumer products drawn from COICOP Division 01.2 (Non-alcoholic beverages), sourced from Zenodo. The dataset spans seven subclasses at the most granular level of the COICOP 2018 hierarchy, with class sizes ranging from 15 to 189 products. The candidate labels for each classifier are the seven subclass names (e.g., "Coffee and coffee substitutes," "Water," "Soft drinks"), optionally accompanied by their official COICOP definitions and an opt-out option ("None of the above").

The baseline classifier is BART-large-MNLI (M. Lewis et al., 2019), a 400M-parameter seq2seq model fine-tuned on the MultiNLI corpus (Williams et al., 2018). BART performs zero-shot classification by computing the entailment probability between the input text (premise) and a hypothesis of the form "This text is about *label*." Inference runs on CPU using the HuggingFace `transformers` library.

The treatment classifier uses `ollama-classifier` with Qwen2.5 3B-Instruct (Yang et al., 2024) served through Ollama (Ollama Contributors, 2023) on a local machine. Classification uses the `classify` method, which combines constrained output generation with multi-call confidence scoring to produce both a predicted label and calibrated probability estimates over all candidate classes.

The second baseline is scikit-llm (Kondrashchenko & Kostromin, 2023), a zero-shot classifier with a scikit-learn-compatible interface. We use its `ZeroShotGPTClassifier` configured to issue requests against the same locally-hosted Qwen2.5 3B-Instruct model, redirected to Ollama through its OpenAI-compatible (`/v1`) endpoint. This ensures that all three classifiers operate on identical input data and share the same underlying language model, isolating the effect of the classification strategy. Unlike `ollama-classifier`, scikit-llm is a label-only classifier: it returns the predicted class but does not expose calibrated confidence scores or a full probability distribution over the candidate classes.

We evaluate eight variations: two for BART (names only, names with opt-out), four for `ollama-classifier` (names only, names with opt-out, names with definitions, names with definitions and opt-out), and two for scikit-llm (names only, names with opt-out). This design allows us to isolate the effects of (i) the underlying model, (ii) the opt-out mechanism, and (iii) the inclusion of subclass definitions as additional context.

We report overall accuracy, macro-averaged precision, recall, and F1-score, the percentage of items classified (excluding opt-outs), accuracy on classified items only, and wall-clock processing time.

6.4.2 Results

Table 2 summarizes the results across all eight variations.

Table 2: Zero-shot classification results on COICOP Division 01.2 (637 products, 7 subclasses). BART uses NLI-based zero-shot classification (M. Lewis et al., 2019); Ollama uses the `classify` method (constrained generation with multi-call confidence scoring) with Qwen2.5 3B-Instruct (Yang et al., 2024); scikit-llm (Kondrashchenko & Kostromin, 2023) uses `ZeroShotGPTClassifier` against the same model through Ollama's OpenAI-compatible endpoint. Macro-averaged metrics are computed over the seven subclasses. "Classif." is the percentage of items assigned a class label (i.e., excluding opt-outs). Wall-clock time is reported in seconds.

Variation	**Acc.** (%)	**Prec.** (macro)	**Rec.** (macro)	**F1** (macro)	**Classif.** (%)	**Time** (s)
BART (names only)	40.8	0.548	0.485	0.445	—	520.4
BART (names + opt-out)	39.4	0.549	0.495	0.454	94.2	610.7
Ollama (names only)	74.3	0.704	0.700	0.689	—	2150.7
Ollama (names + opt-out)	71.9	0.683	0.704	0.678	96.4	2452.6
Ollama (names + desc.)	70.8	0.684	0.704	0.677	—	2207.7
Ollama (desc. + opt-out)	69.1	0.692	0.719	0.685	93.7	2486.6
scikit-llm (names only)	73.5	0.690	0.689	0.667	—	204.5
scikit-llm (names + opt-out)	68.0	0.631	0.701	0.651	89.6	199.9

Model comparison. `ollama-classifier` with Qwen2.5 3B-Instruct substantially outperforms BART-large-MNLI across all configurations. The best Ollama variation (names only, no opt-out) achieves 74.3% overall accuracy compared to 40.8% for the best BART variation, representing a 33.5 percentage-point improvement. scikit-llm, which drives the same Qwen2.5 3B-Instruct model through a single constrained-generation call rather than multi-call scoring, achieves 73.5% accuracy—within one percentage point of `ollama-classifier`. This confirms that the generative model's advantage over the NLI baseline stems from its superior ability to reason over fine-grained semantic distinctions rather than from the scoring protocol. The comparable accuracy of scikit-llm, however, comes at the cost of discarding confidence information: because it returns only the predicted label, the probability-based triage strategies analyzed below are inapplicable to it. In terms of runtime, scikit-llm is the fastest classifier (≈200 s, roughly 10× faster than the multi-call Ollama variations at ≈2150–2490 s), reflecting its single-call design, while BART runs on CPU at ≈520–610 s.

Opt-out behavior. When the opt-out option is enabled, BART opts out of 37 products (5.8%), Ollama opts out of 23 products (3.6%), and scikit-llm opts out of 66 products (10.4%). Among classified items, Ollama's accuracy rises from 74.3% to 74.6% with opt-out enabled, and scikit-llm's rises from 73.5% to 75.8%, suggesting that the generative models correctly identify their least-confident predictions. BART's accuracy on classified items similarly improves from 40.8% to 41.8%, but the magnitude of improvement is far smaller, consistent with BART's lower overall discriminative capability. scikit-llm's more aggressive abstention—nearly three times Ollama's opt-out rate—reduces its overall accuracy to 68.0% but yields the highest on-classified accuracy of the three classifiers.

Effect of subclass definitions. Including official COICOP definitions alongside subclass names does not improve classification performance for Ollama. The names-only variation (74.3%) outperforms the names-with-descriptions variation (70.8%) by 3.5 percentage points, indicating that definitional context actively harms performance in this domain. This suggests that for Division 01.2—where subclass names such as "Coffee and coffee substitutes" and "Tea, máté and other plant products for infusion" are already highly distinctive—additional definitional context provides marginal benefit and may occasionally introduce noise. This finding aligns with the results of Nunes et al. (2025), who report that classification-by-definition is most beneficial at higher COICOP levels where class boundaries are less clear.

Confidence and calibration. Because scikit-llm returns only the predicted label, the following analysis covers the six confidence-bearing variations of BART and `ollama-classifier`. Table 3 summarizes the relationship between classifier confidence and prediction correctness. Both models exhibit a statistically significant positive correlation between the confidence score assigned to the predicted label and whether the prediction is correct. BART's Pearson correlation coefficients range from $r = 0.347$ to $r = 0.351$, while Ollama's range from $r = 0.402$ to $r = 0.452$ (all $p < 0.001$). As a stronger, directional test, one-tailed Welch's t-tests confirm that incorrect predictions carry significantly lower mean confidence than correct ones in every variation ($t = 8.76$–9.91, all $p < 0.001$). Together, the correlations and t-tests establish that confidence discriminates correctness rather than being merely a monotonic artifact of the score distribution.

The two models differ markedly in their confidence distributions. Ollama's mean confidence for correct predictions is 0.954–0.959 across variations, compared to 0.509–0.532 for BART. Notably, Ollama's mean confidence for *incorrect* predictions (0.804–0.820) exceeds BART's mean confidence for *correct* predictions. This indicates that Ollama's probability estimates are compressed into a narrow high-confidence band, making absolute threshold-based triage less effective.

The mean score gap—the difference between the two highest class probabilities—further illuminates this pattern. For Ollama, the mean gap on incorrect predictions is 0.85–0.86 (median ≈ 0.99), showing a strong preference for the wrong label over the correct one. BART's gap on incorrect predictions is 0.23–0.25, reflecting a more diffuse probability distribution where the model is less decisive. These findings suggest that confidence thresholds are informative for both models, but finer-grained triage strategies—such as examining the score gap or the probability assigned to the ground-truth class—may be more effective for Ollama than a simple threshold on the predicted label's confidence.

Figure 1 visualizes the confidence distributions underlying these statistics. The compression of Ollama's probabilities is immediately apparent: even its incorrect predictions typically carry confidence above 0.80, overlapping with BART's *correct* predictions. This compression limits the granularity of absolute thresholding for Ollama, whereas BART's broader, better-separated distributions make a single confidence cutoff a more effective triage signal.

The accuracy–coverage trade-off implied by thresholding is quantified in Figure 2. The selective-accuracy curves (left) show that, for both models, restricting to high-confidence predictions raises accuracy as the threshold increases; the coverage curves (right) show the corresponding fraction of items retained. BART offers a smoother trade-off, steadily gaining accuracy as coverage falls. Ollama's accuracy also rises with the threshold, but because its confidence is concentrated above 0.9, large gains in selective accuracy are achieved only by sacrificing substantial coverage. For practical deployment, this means that ranking items by confidence (or by score gap) and reviewing the lowest-ranked tail is more effective for Ollama than applying a hard probability cutoff.

Practical implications. These results demonstrate that a compact 3B-parameter model served locally through Ollama can match or exceed the zero-shot classification performance of a dedicated NLI model on a fine-grained product taxonomy task—without requiring fine-tuning, API access, or data to leave the user's machine. The constrained output generation mechanism

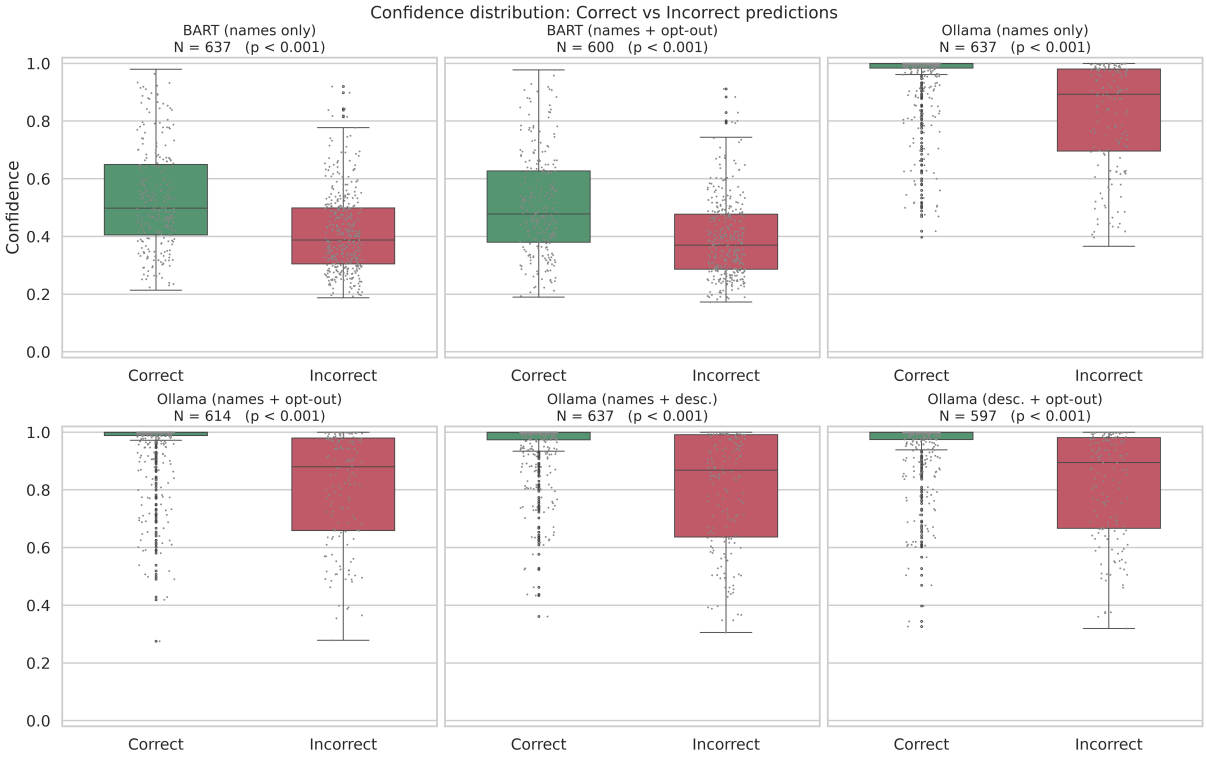
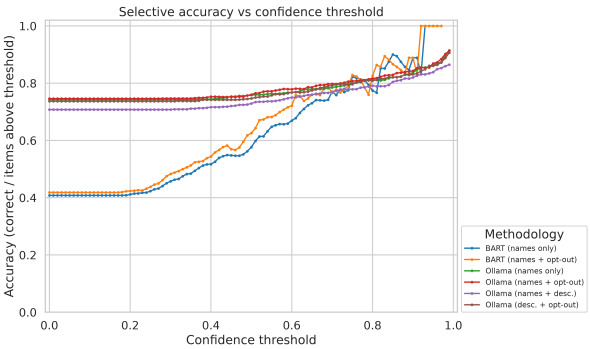
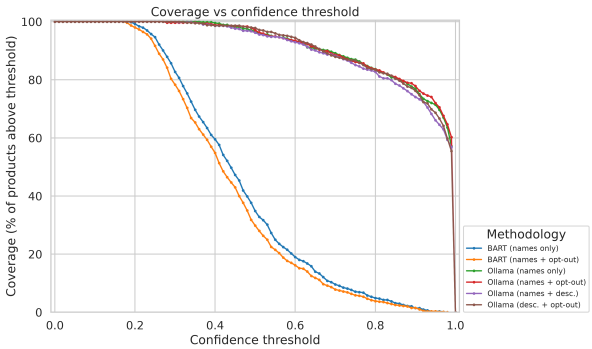


Figure 1: Distribution of predicted-label confidence for correct versus incorrect predictions across the six confidence-bearing variations. For both models, incorrect predictions concentrate at lower confidence, but Ollama's distributions are compressed in a high-confidence band (median ≥ 0.95 even for most incorrect cases).



(a) Selective accuracy: accuracy among items whose confidence meets or exceeds the threshold.



(b) Coverage: fraction of items retained at each confidence threshold.

Figure 2: Accuracy–coverage trade-off under confidence thresholding. BART exhibits a smoother, better-separated trade-off; Ollama's high-confidence compression means coverage remains high until thresholds approach 0.9, so ranking-based triage outperforms hard cutoffs.

Table 3: Confidence analysis across classification variations. "Conf (correct)" and "Conf (incorrect)" report the mean predicted-label confidence for correct and incorrect predictions, respectively. "Corr. r" is the Pearson correlation between predicted-label confidence and correctness. "Score gap" is the mean difference between the highest and second-highest class probabilities. "Welch t" is the one-tailed Welch's t-statistic testing whether incorrect predictions have lower mean confidence than correct ones (all $p < 0.001$). scikit-llm is omitted because it is a label-only classifier that does not produce confidence scores.

Variation	**Conf. (correct)**	**Conf. (incorrect)**	**Corr. r**	**Score gap**	**Welch t**
BART (names only)	0.532	0.414	0.347	0.245	8.93
BART (names + opt-out)	0.509	0.392	0.351	0.228	8.80
Ollama (names only)	0.954	0.820	0.402	0.862	8.76
Ollama (names + opt-out)	0.959	0.810	0.437	0.864	9.16
Ollama (names + desc.)	0.956	0.804	0.452	0.848	9.91
Ollama (desc. + opt-out)	0.954	0.815	0.422	0.861	8.91

ensures that all predictions are valid class labels, eliminating the post-processing required by unconstrained LLM outputs. The opt-out mechanism provides a principled way to flag uncertain predictions for human review, a critical feature for production deployment in official statistics. Crucially, the calibrated confidence scores are what distinguish `ollama-classifier` from a faster label-only classifier such as scikit-llm: although scikit-llm matches the generative model's accuracy at a fraction of the runtime, the absence of confidence information precludes the selective-accuracy triage shown in Figure 2, which lets analysts trade coverage for precision automatically.

6.4.3 Limitations of the Evaluation

Several caveats apply. The evaluation covers only one COICOP division (01.2, Non-alcoholic beverages), and performance may differ at other divisions where class boundaries are more ambiguous. We evaluate a single Ollama model (Qwen2.5 3B-Instruct); larger models may achieve higher accuracy, while smaller models may fall below the observed threshold. The BART baseline uses CPU inference, and GPU acceleration could narrow the runtime gap. Finally, we do not include a fine-tuned classifier baseline; such a comparison would require task-specific training data and is left to future work. Additionally, the scikit-llm comparison is confined to accuracy and opt-out behavior: because it is a label-only classifier, it could not be included in the confidence and calibration analysis, so the relative calibration of its predictions remains unassessed.

7 Discussion

7.1 Design Decisions

Several design decisions distinguish `ollama-classifier` from alternative approaches:

Local-first inference. By defaulting to Ollama for local model serving, the library prioritizes privacy and cost-effectiveness. While cloud-based APIs offer access to larger models, the trade-off is often unacceptable in sensitive domains. The support for multiple backends (vLLM, SGLang, llama.cpp) ensures that users can choose the inference engine that best matches their hardware and deployment requirements.

No fine-tuning. The library deliberately avoids any fine-tuning step, relying entirely on

prompting and retrieval augmentation. This design choice maximizes flexibility: users can switch models, add or remove classes, and adapt to new domains without retraining. The trade-off is that fine-tuned models may achieve higher accuracy on specific tasks, but at the cost of reduced generality and increased computational requirements.

Constrained output. Enforcing valid class labels through prompt design and, where available, JSON schema constraints is essential for production deployment. Unconstrained LLM outputs are prone to hallucination, verbosity, and formatting inconsistencies that would break downstream processing pipelines.

7.2 Limitations

Several limitations should be acknowledged. First, the quality of classification depends on the underlying model's capabilities. While recent open-source models have improved dramatically, smaller models (below 3B parameters) may struggle with complex or ambiguous classification tasks. The results reported by Nunes et al. (2025) show that models with fewer than 1.5B parameters achieve only marginal classification accuracy, suggesting a practical lower bound for effective deployment.

Second, bias and fairness concerns apply to LLM-based classification (Gallegos et al., 2024). The models' pre-training data may encode societal biases that affect classification outcomes, particularly when classifying texts related to demographic, economic, or cultural categories. Users should be aware of these risks and consider bias auditing tools (Yuan et al., 2026) when deploying `ollama-classifier` in sensitive applications.

Third, the multi-call scoring strategy trades computational cost for calibrated confidence estimates. For classification tasks with many candidate labels, the N-call approach may be prohibitively expensive, and users may need to rely on the faster generate-only strategy.

Finally, while the GUI application significantly lowers the barrier to entry, it currently operates as a single-user desktop application. Multi-user, server-based deployments would require additional infrastructure that is beyond the current scope.

8 Conclusion

We have presented `ollama-classifier`, an open-source Python library that enables zero-shot and few-shot text classification using locally-hosted Large Language Models. The library provides constrained output generation, confidence scoring, batch processing, and multi-backend support, making advanced LLM-based classification accessible without fine-tuning, API keys, or internet connectivity. An R port, `rollama-classifier` (Palumbo, 2026b), further extends this accessibility to the R community.

The companion GUI application extends this accessibility to non-technical users, providing an intuitive four-tab workflow for the complete classification pipeline—from backend configuration and data loading to schema definition and results review. This combination of a programmatic library and a graphical interface positions `ollama-classifier` as a practical tool for bridging the gap between the capabilities of modern LLMs and the operational needs of domain practitioners in fields such as official statistics, economic research, and data governance.

Future work will focus on expanding the retrieval-augmented classification pipeline within the library, improving confidence calibration through temperature scaling and conformal prediction methods, and developing a web-based interface to support collaborative, multi-user classification workflows.

References

Abdullahi, T., Singh, R., & Eickhoff, C. (2024). Retrieval augmented zero-shot text classification. *Proceedings of the 2024 ACM SIGIR International Conference on Theory of Information Retrieval*, 195–203. https://doi.org/10.1145/3675118.3675148

Extends retrieval-augmented classification to the zero-shot setting, closely aligning with ollama-classifier's ability to classify into arbitrary taxonomies without training data. The library's approach of retrieving exemplars from a synthetic knowledge base reflects the methodology explored in this work.

Amershi, S., Begel, A., Bird, C., DeLine, R., Gall, H., Kamar, E., Nagappan, M., Nushi, B., & Zimmermann, T. (2019). Software engineering for machine learning: A case study. *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice*, 291–300. https://doi.org/10.1109/ICSE-SEIP.2019.00042

Discusses the challenges of building ML-powered applications and the gap between ML models and end-users. Directly supports the motivation for the ollama-classifier GUI app, which aims to make LLM-based classification accessible to non-technical users such as domain experts and statisticians without requiring programming expertise.

Berki, M., Andicsova, V., & Oravec, M. (2025). NLP-enhanced inflation measurement using BERT and web scraping. *Frontiers in Artificial Intelligence*, *8*, 1520659. https://doi.org/10.3389/frai.2025.1520659

Demonstrates the application of transformer models (BERT) to inflation measurement via product classification. This work represents the traditional fine-tuning approach that ollama-classifier aims to supplant with its retrieval-augmented, no-fine-tuning methodology, providing a clear comparative baseline for the domain application.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, *33*, 1877–1901. https://arxiv.org/abs/2005.14165

Foundational paper demonstrating GPT-3's few-shot learning capabilities without fine-tuning. Establishes the theoretical basis for using LLMs as classifiers through in-context learning, which is the core paradigm that ollama-classifier implements with smaller, locally-runnable models via Ollama.

Cavallo, A. (2013). Online and official price indexes: Measuring argentina's inflation. *Journal of Monetary Economics*, *60*(2), 152–165. https://doi.org/10.1016/j.jmoneco.2012.10.003

Establishes the economic motivation for automated product classification by demonstrating the viability of online price data for computing inflation indicators. Provides the broader context that motivates ollama-classifier as a practical tool for National Statistical Institutes needing scalable classification of web-scraped products.

Cavallo, A. (2017). Are online and offline prices similar? evidence from large multi-channel retailers. *American Economic Review*, *107*(1), 283–303. https://doi.org/10.1257/aer.20151526

Provides evidence that online prices reliably track offline market dynamics, further legitimizing the use of web-scraped data for economic statistics. Supports the case for tools like ollama-classifier that enable automatic classification of online product data at scale.

Chen, J., Zhang, Q., Zheng, X., Zhu, G., Deng, Z., Cui, Y., Xiao, W., Liu, T., Zhang, P., Yang, X., et al. (2024). Gte: General text embeddings from ALIBABA. https://huggingface.co/Alibaba-NLP/gte-Qwen2-7B-instruct

The GTE embedding family (gte-multilingual-base, gte-Qwen2-7B-instruct) achieved the highest retrieval presence in the RAC evaluation (90.7%). ollama-classifier supports these models in its retrieval pipeline, enabling users to leverage state-of-the-art semantic search for their classification tasks.

CVxTz. (2023). *Llmclassifier: Building reliable text classification pipelines with LLMs*. https://github.com/CVxTz/llmclassifier

An open-source Python library designed to build text classification pipelines using LLMs. Represents the competitive landscape and related work for ollama-classifier. Unlike ollama-classifier, which focuses on local inference via Ollama and retrieval augmentation, this library generally targets API-based models, highlighting ollama-classifier's unique value proposition of privacy and offline capability.

DeepSeek-AI. (2025). DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*. https://arxiv.org/abs/2501.12948
DeepSeek-R1 models demonstrated a distinctive conservative classification strategy with well-calibrated uncertainty estimation and high accuracy on classified items. ollama-classifier supports these models, and their opt-out behavior is particularly relevant for human-in-the-loop scenarios targeted by the library's GUI app.

Gallegos, A., et al. (2024). Bias and fairness in large language models: A survey. *arXiv preprint arXiv:2309.00770*. https://doi.org/10.48550/arXiv.2309.00770
Surveys bias and fairness issues in Large Language Models. Crucial for the limitations section of the research paper, as using LLMs as classifiers via ollama-classifier risks amplifying societal biases present in the pre-training data, which must be acknowledged when classifying economic or demographic data.

Gilardi, F., Alizadeh, M., & Kubli, M. (2023). ChatGPT outperforms crowd-workers for text-annotation tasks. *Proceedings of the National Academy of Sciences*, *120*(7), e2215907120. https://doi.org/10.1073/pnas.2215907120
Demonstrates that LLMs can match or exceed human crowd-workers for text classification tasks. This finding directly supports ollama-classifier's premise: that locally-run LLMs, especially when augmented with retrieval, can serve as reliable classifiers, reducing or eliminating the need for manual annotation.

Golino, H. (2023). *Llmclassificr: Toolkit per classificazione testuale con large language models*. https://github.com/GoBru/llmClassificR
An R package for text classification using LLMs. Serves as cross-language related work, demonstrating that the demand for LLM-based classification tools spans beyond the Python ecosystem. It reinforces the need for ollama-classifier's specialized focus on retrieval-augmented, constrained-generation workflows in Python.

Hakimi, V., Bowen, L., et al. (2026). Do we still need humans in the loop? comparing human and LLM annotation in active learning. *arXiv preprint arXiv:2601.09346*. https://doi.org/10.48550/arXiv.2601.09346
Compares human annotators with LLMs within an active learning framework. Provides empirical context for the research paper's argument that tools like ollama-classifier should be viewed as assistants that reduce human labeling burden (e.g., via confident auto-labeling and uncertain opt-outs) rather than full replacements for domain experts.

He, X., Zhao, K., & Chu, X. (2021). AutoML: A survey of the state-of-the-art. *Knowledge-Based Systems*, *228*, 107243. https://doi.org/10.1016/j.knosys.2021.107243
AutoML aims to automate the ML pipeline, making it accessible to non-experts. The ollama-classifier GUI shares this philosophy, abstracting the complexity of RAC—including retrieval configuration, embedding selection, and prompt engineering—behind an intuitive interface, analogous to how AutoML tools hide hyperparameter tuning and model selection.

Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., Chaplot, D. S., Casas, D. L., Hanna, E., et al. (2023). Mistral 7B. *arXiv preprint arXiv:2310.06825*. https://arxiv.org/abs/2310.06825
Mistral 7B achieved the highest overall classification accuracy in the RAC evaluation despite its compact size. Its small footprint makes it ideal for local deployment on consumer hardware via Ollama, aligning with ollama-classifier's goal of accessible, efficient classification.

Kondrashchenko, I., & Kostromin, O. (2023). *Scikit-llm: Scikit-learn meets large language models*. Linz, Austria, beastbyte.ai. https://github.com/iryna-kondr/scikit-llm

Lazaridou, A., Kobyzev, I., Mahowald, K., Mordatch, I., Niekum, S., et al. (2023). Towards AI that can solve science: A community-driven perspective. *arXiv preprint arXiv:2308.11016*. https://arxiv.org/abs/2308.11016
Discusses the importance of democratizing access to AI tools for domain experts who are not ML specialists. Relevant to the ollama-classifier GUI's goal of enabling non-technical users—such as economists, statisticians, and data officers—to leverage state-of-the-art LLM classification without writing code.

Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V., & Zettlemoyer, L. (2019). BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *arXiv preprint arXiv:1910.13461*. https://arxiv.org/abs/1910.13461
Introduces BART, a denoising autoencoder for pre-training sequence-to-sequence models. BART-large fine-tuned on MNLI (facebook/bart-large-mnli) serves as the zero-shot classification baseline in our evaluation, framing classification as natural language inference.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, *33*, 9459–9474. https://arxiv.org/abs/2005.11401
Seminal paper introducing Retrieval-Augmented Generation (RAG), which grounds LLM outputs in externally retrieved evidence to improve factuality and stability. The ollama-classifier library builds upon this paradigm, using retrieval to condition the generative classification step, improving both accuracy and interpretability over pure prompting approaches.

Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., & Neubig, G. (2023). Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, *55*(9), 1–35. https://doi.org/10.1145/3560815
Surveys prompting methodologies directly relevant to how ollama-classifier instructs LLMs to perform classification. The library's prompt templates for classification-by-example and classification-by-definition are informed by the prompting strategies catalogued in this survey.

Meta AI. (2024). The LLaMA 3 herd of models. *arXiv preprint arXiv:2407.21783*. https://arxiv.org/abs/2407.21783
LLaMA 3 represents the latest generation of Meta's open models and is a primary model family used with ollama-classifier. LLaMA 3 8B demonstrated strong overall classification performance in the RAC evaluation, and these models are readily available through Ollama for local inference in the library's pipeline.

Mu, Y., & Ziebart, B. (2024). Navigating prompt complexity for zero-shot classification: A study of large language models in computational social science. *arXiv preprint arXiv:2305.14310*. https://doi.org/10.48550/arXiv.2305.14310
Investigates how prompt complexity affects zero-shot classification performance by LLMs. This is directly relevant to ollama-classifier, as the library's effectiveness heavily depends on how well its prompt templates (e.g., classification-by-example vs. classification-by-definition) structure the retrieved context for the LLM.

Nunes, I., Palumbo, L., & Bacco, L. (2025). Classification at scale: A retrieval-augmented classification framework for COICOP 2018 consumer products. *Manuscript under review*. https://github.com/paluigi/ollama-classifier
Foundational work by the same author of ollama-classifier, describing the RAC framework that the library implements. This paper demonstrates the methodology on COICOP

product classification, achieving up to 0.80 accuracy at the finest granularity level. The ollama-classifier library is the software realization of this framework, packaged as a general-purpose, reusable tool.

Ollama Contributors. (2023). *Ollama: Run large language models locally*. https://ollama.com
Ollama is the underlying runtime used by ollama-classifier to serve open-source LLMs (e.g., LLaMA, Mistral, Qwen, DeepSeek) on consumer hardware. By eliminating dependency on cloud APIs, Ollama provides the privacy, cost-effectiveness, and offline capability that are central to the library's value proposition.

Ollama Team. (2023). *Ollama python library*. https://github.com/ollama/ollama-python
The official Python client library that ollama-classifier wraps to interface with the local Ollama server. It handles API communication, enabling the classification library to send prompts and receive structured outputs seamlessly without requiring users to write low-level HTTP calls.

OpenAI. (2023). Gpt-4 technical report. https://doi.org/10.48550/arXiv.2303.08774
While ollama-classifier focuses on local, open-source models, GPT-4 serves as the proprietary upper-bound benchmark for zero-shot and few-shot reasoning tasks. It contextualizes the performance trade-offs inherent in using smaller, locally-hosted models via Ollama for privacy and cost-efficiency.

Palumbo, L. (2026a). *Ollama-classifier-gui: A desktop gui for retrieval-augmented classification with local large language models*. https://github.com/paluugi/ollama-classifier-gui
The companion desktop application of the ollama-classifier library, built with Flet. It exposes the full classification workflow (backend configuration, data loading, schema definition, and results review) through a point-and-click four-tab interface, enabling non-technical users to leverage retrieval-augmented LLM classification without writing code. Publicly released on GitHub and PyPI.

Palumbo, L. (2026b). *Rollama-classifier: An r port of ollama-classifier for text classification with constrained output and confidence scoring*. https://github.com/paluigi/rollama-classifier

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language models are unsupervised multitask learners* (tech. rep.). OpenAI. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf
Introduces GPT-2, demonstrating that language models can perform zero-shot task transfer without explicit supervision. This is the foundational paper that initiated the paradigm of using generative models as classifiers, which is the core theoretical premise underlying the ollama-classifier library.

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 3982–3992. https://doi.org/10.18653/v1/D19-1410
SentenceTransformers is the framework underlying many embedding models used in ollama-classifier's retrieval pipeline. This paper introduces the Siamese BERT architecture for producing semantically meaningful sentence embeddings, which are essential for the similarity-based retrieval step in the RAC framework.

Schroeder, H., Roy, D., & Kabbara, J. (2025). Just put a human in the loop? investigating LLM-assisted annotation for subjective tasks. *Findings of the Association for Computational Linguistics: ACL 2025*, 25771–25795. https://doi.org/10.18653/v1/2025.findings-acl.1323
Examines the efficacy and pitfalls of LLM-assisted annotation. Highly relevant to the planned GUI app for ollama-classifier, as it highlights the necessity of human-in-the-loop designs—aligning perfectly with the library's 'opt-out' mechanism where the model abstains rather than forcing an incorrect label.

Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., et al. (2023). LLaMA 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*. https://arxiv.org/abs/2307.09288
LLaMA 2 is a key model family supported by Ollama and commonly used with ollama-classifier. This paper establishes the open-source foundation model ecosystem that makes local LLM deployment feasible, directly enabling the library's operation without proprietary APIs.

United Nations Statistics Division. (2018). Classification of individual consumption according to purpose (COICOP) 2018. https://unstats.un.org/unsd/classifications/Family/Detail/21
COICOP 2018 is the international taxonomy used as the primary validation use case for ollama-classifier. This reference provides the official definitions and hierarchical structure that the library uses to construct synthetic knowledge bases and perform classification, demonstrating its applicability to real-world statistical classification tasks.

Vasquez Ferrer, R., Turgut, D., et al. (2026). When can we trust LLM graders? calibrating confidence for automated assessment. *arXiv preprint arXiv:2603.29559*. https://doi.org/10.48550/arXiv.2603.29559
Explores calibrating confidence scores for LLM automated assessment. Directly related to the 'confidence scoring' feature of ollama-classifier, as reliable confidence metrics are essential for the GUI application to triage classifications, automatically accepting high-confidence labels and routing low-confidence ones to human reviewers.

Wang, L., Yang, N., Huang, X., Jiao, B., Yang, L., Jiang, R., Majumder, R., & Wei, F. (2022). Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*. https://arxiv.org/abs/2212.03533
Introduces the E5 embedding family (multilingual-e5-base, multilingual-e5-large), which are supported by ollama-classifier for the retrieval component. These embeddings provide the multilingual semantic similarity capabilities required for cross-lingual classification tasks.

Wang, Z., Xie, Q., Feng, Y., Ding, Z., Yang, Z., & Xia, R. (2024). Is ChatGPT a good sentiment analyzer? a preliminary study. *arXiv preprint arXiv:2304.04339*. https://doi.org/10.48550/arXiv.2304.04339
Evaluates ChatGPT's performance as a sentiment analyzer. While focused on proprietary models, this study is representative of the broader research trend of repurposing generative LLMs as discriminative classifiers—the exact paradigm shift that ollama-classifier operationalizes for local, open-source models.

Williams, A., Nangia, N., & Bowman, S. R. (2018). A broad-coverage challenge corpus for sentence understanding through inference. *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 1112–1122. https://doi.org/10.18653/v1/N18-1101
Introduces the Multi-Genre Natural Language Inference (MNLI) corpus used to fine-tune BART for the zero-shot classification baseline. The NLI formulation—mapping classification to textual entailment—is the standard approach for BART-based zero-shot classification.

Yang, A., Yang, B., Hui, B., Zheng, B., Yu, B., et al. (2024). Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*. https://arxiv.org/abs/2412.15115
Qwen2.5-32B achieved the highest accuracy in the RAC evaluation across all COICOP levels using the classification-by-definition approach. Available through Ollama, it represents a strong choice for ollama-classifier users who prioritize classification precision and can allocate sufficient hardware resources.

Yu, G., Liu, L., Jiang, H., Shi, S., & Ao, X. (2023). Retrieval-augmented few-shot text classification. *Findings of the Association for Computational Linguistics: EMNLP*, 6721–6735. https://arxiv.org/abs/2308.13314

Directly relevant to the classification methodology implemented in ollama-classifier. Demonstrates that combining nearest-neighbor retrieval with few-shot prompting improves text classification without fine-tuning, which is the core mechanism the library implements using Ollama-based models.

Yuan, Z., et al. (2026). Bias ahead: Sensitive prompts as early warnings for fairness in large language models. *arXiv preprint arXiv:2604.05575*. https://doi.org/10.48550/arXiv.2604.05575

Proposes using sensitive prompts as early warning systems for fairness issues in LLMs. Relevant to the ollama-classifier framework as a potential future diagnostic tool to test whether the retrieval-augmented classification pipeline introduces or mitigates biases when categorizing products across different languages or regions.

Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., et al. (2023). A survey of large language models. *arXiv preprint arXiv:2303.18223*. https://arxiv.org/abs/2303.18223

Comprehensive survey covering the capabilities, architectures, and training paradigms of modern LLMs. Provides the theoretical background for understanding why LLMs can serve as effective classifiers, supporting the paper's argument that generative models can be repurposed for discriminative tasks through appropriate prompting and retrieval augmentation.